- Plan
- Généralités sur la POO
- La notion des classes
- Définir une classe
- Appeler une classe
- Les attributs d'une classe
- Le constructeur par défaut
- Les méthodes d'une classe

Généralités sur la POO

Généralité

- La programmation orientée objet (POO), ou programmation par objet, est un paradigme de programmation informatique élaboré par les Norvégiens Ole-Johan Dahl et Kristen Nygaard au début des années 1960 et poursuivi par les travaux d'Alan Kay dans les années 1970.
- Il consiste en la définition et l'interaction de briques logicielles appelées objets ; un objet représente un concept, une idée ou toute entité du monde physique,comme une voiture, une personne ou encore une page d'un livre.
- Un objet possède une structure interne et un comportement, et il sait interagir avec ses pairs. Il s'agit donc de représenter ces objets et leurs relations l'interaction entre les objets via leurs relations permet de concevoir et réaliser les fonctionnalités attendues, de mieux résoudre le ou les problèmes.
- C++ est un langage de programmation orientée objets. Les objets sont omniprésents en C++, char, int, float,list ...(qui sont alors dits natifs, c'est à dire par défaut dans C++), mais est qu'on peut créer des objets personnalisés selon nos besoin ?

La notion des classes

Classe

Une classe représente un type personnalisé et complexe.
Une classe est un ensemble d'objets qui partagent les mêmes propriétés (attributs) et les mêmes fonctions (méthodes).
Les fonctions qui agissent sur les objets d'une classe sont appelées méthodes.

classe = Attributs + méthodes

Définir une classe

Définir une classe

Pour définir une classe, on commence par l'utilisation du mot clé class, suivie par le nom de la classe.
Ensuite, on définit les attributs de la classe Enfin, on définit l'ensemble des méthodes qui agissent sur la classe.
Parmi les méthodes on trouve le constructeur qui porte le même nom que le nom de la classe, le mot clé this désigne l'objet courant. Cet objet représente l'instance courante de la classe elle-même.

Syntaxe

```
class NomDeLaClasse {
mode_d'encapsulation_desAttributs:
        type Attr 1;
        .....
        type AttrN;
mode_d'encapsulation_desMéthodes:
        typeDeRetour methode1(paramètres);
                .....
        typeDeRetour methodeM(paramètres);
};
```

Défintions

Le C++ implémente l'encapsulation en permettant de déclarer les membres d'une classe avec l'un des mots réservés public, private et protected. Ainsi, lorsqu'un membre est déclaré:
public, il sera accessible depuis n'importe quelle fonction.
private, il sera uniquement accessible d'une part, depuis les fonctions qui sont membres de la classe et, d'autre part, depuis les fonctions et classes autorisées explicitement par la classe (par l'intermédiaire du mot réservé friend). Ces dernières fonctions/classes sont appelées fonctions/classes amies de la classe.
protected, il aura les mêmes restrictions que s'il était déclaré private, mais il sera en revanche accessible par les classes filles.

Exemple

Classe Point.h

```
#include <iostream>
#include<math.h>
using namespace std;
class Point
{
  private:
  float x;
  float y;
  public:
  Point(float _x,float _y);
  void set_x(float _x);
  void set_y(float _y);
  float get_x();
  float get_y();
  void afficherPoint();
  float distance(Point Q);
};
```

Implémentation de la classe Point.cpp

```
#include <iostream>
#include<math.h>
#include "Point.h"
using namespace std;
Point::Point(float _x,float _y){ this->x=_x;this->y=_y;}
void Point::set_x(float _x) {x=_x; };
void Point::set_y(float _y) {this->y=_y; };
float Point::get_x() {return this->x; };
float Point::get_y() {return this->y; };
void Point::afficherPoint(){cout<<"x ="<<this->x<<" y="<<this->y<<endl; };
float Point::distance(Point Q) {return pow(pow((Q.x-x),2)+pow((Q.y-y),2),1/2); };
```


Exemples

main.cpp

```
#include <iostream>
#include<math.h>
#include "Point.h"
using namespace std;
int main(int argc, char** argv)
{

  Point P(2,3);
  P.afficherPoint();
  return 0;
}
```

Exemple 1

.....

Exemple 2

.....

Exemple 3

.....

Les attributs d'une classe

Les attributs d'une classe

Les attributs d'une classe constituent les caractéristiques des objets générés à partir de cette classe Un attribut est caractérisé par :

- un nom
- une valeur obligatoire (qui désigne le type de l'attribut)

Les attributs d'un objet sont accessibles et modifiables après la construction de l'objet.
Exemple de la classe Point

```
Point P(2,3); //Appel de constructeur par défaut
P.x=1;
P.y=2;
cout<<P.x; afficher .....
cout<<P.y; afficher .....
```

Le constructeur par défaut

Le constructeur par défaut

On considère l'exemple suivant

```
Class Point:
        Point::Point (float xcoord=0,float ycoord=0){
                this->x = xcoord ;
                 this-> y = ycoord;
                   }
```

Dans cet exemple la méthode Point (qui est le constructeur par défaut) est explicite, elle possède 2 paramètres

```
Point P1(10,20) // création de l'objet P1 sans problème
Point::Point(){this->x=0;this->y=0;};
```

Point P2 #création d'un objet avec un constructeur ayant des valeurs par défaut P2.x=0 et P2.y=0

Les méthodes d'une classe

Pour accéder aux méthodes d'un objet, on indique :
1. le nom de la variable qui fait référence à cet objet
2. un point
3. le nom de la méthode

Exemple

```
Point P; # on suppose que le constructeur par défaut est déjà implémenté
P.initialiserPoint(8,9)
P.afficherPoint()
```

Le résultat d'exécution est : .....

Les méthodes d'une classe

Les méthodes ou les opérations d'une classe décrivent le comportement des objets générées à partir de cette classe.
Les méthodes assurent l'interaction entre les différents objets à travers l'échange de messages
Une méthode représente une fonction qui permet de réaliser une ou plusieurs tâches Pour définir une méthode, il faut :

- indiquer son nom.
- indiquer les arguments entre des parenthèses. Le premier argument d'une méthode doit être self.

Exemple

```
class Point {
/* Modélise les points d'un plan muni d'un repère */
        private :
        float x;
        float y;
        #Défintion de la méthode initialiserPoint()
        void Point::initialiserPoint (float a,float b)
                {
                  this->x=a;
                  this->y=b;
                }
```